

📌 Assignment: University System (Code First + Relationships + Loading)

🎯 Objective

Build a University Management System using ASP.NET Core + Entity Framework Core (Code First) while implementing:

CRUD Operations

All types of relationships

Data Loading strategies:

Eager Loading

Lazy Loading (Bonus)

Explicit Loading (Bonus)

□ Scenario

You are developing a system to manage a university, including:

Students

Departments

Courses

Instructors

Enrollments

The system should reflect real-world relationships between these entities.

Data Model Requirements

Entities & Relationships

Student

Id

Name

Age

Email

DepartmentId

Relationship:

Many Students → One Department (Many-to-One)

Many Students ↔ Many Courses (via Enrollment)

Department

Id

Name

☞ Relationship:

One Department → Many Students (One-to-Many)

One Department → One Instructor (Head) (One-to-One)

☐ Course

Id

Title

Credits

☞ Relationship:

Many Courses ↔ Many Students (Many-to-Many via Enrollment)

Many Courses → One Instructor (Many-to-One)

☐ Instructor

Id

Name

Salary

DepartmentId

☞ Relationship:

One Instructor → Many Courses (One-to-Many)

One Instructor ↔ One Department (Head) (One-to-One)

📄 Enrollment (Join Table)

Id

StudentId

CourseId

Grade

☞ Represents:

Many-to-Many between Students and Courses

⚙️ Task Requirements

📄 Setup (Code First)

Create Models

Configure relationships using:

Data Annotations OR Fluent API

Run migrations:

Add-Migration InitialCreate

Update-Database

☐ Implement CRUD

Students

Create / Read / Update / Delete

Courses

Create / Assign Instructor

Departments

Create / Assign Head Instructor

Enrollment

Enroll Student in Course

Assign Grade

☐ Relationship Scenarios (IMPORTANT 🔥)

You must implement:

Get Student with:

Department

Courses

Get Course with:

Instructor

Enrolled Students

Get Department with:

Students

Head Instructor

🚀 Data Loading Requirements

● Eager Loading (Required)

Use `.Include()` and `.ThenInclude()`:

Example:

```
var students = _context.Students
    .Include(s => s.Department)
    .Include(s => s.Enrollments)
    .ThenInclude(e => e.Course)
    .ToList();
```

□ Explicit Loading (Bonus)

```
var student = _context.Students.First();
```

```
_context.Entry(student)
    .Collection(s => s.Enrollments)
    .Load();
```

□ Lazy Loading (Bonus)

Enable Lazy Loading using proxies

Use virtual navigation properties

□ Testing Scenarios

Student enrolled in multiple courses

Course has multiple students

Department has head instructor

Data loads correctly using:

Eager loading

Explicit loading